KubSTU CTF Writeup

costavinc

01/05/2026

Contents

1 [Crypto] – Base

1.1 Challenge Description

We intercepted a strange message. It seems to be encoded using a popular method. Help us figure out what it says.

1.2 Enumeration & Static Analysis

As we are presented with a txt file, we can easily analyze its contents:

```
file ./Base.txt
cat ./Base.txt
```

Listing 1: initial file checking

The output clearly shows a string encoded in base64:

```
S3ViU1RVKGI0czNfNjRfMXNfdGhlX2JhNWk1KQ==
```

Listing 2: cat output

1.3 Exploitation / Solution Path

Since we are presented with a base64 encoded string, we can easily decode it in Bash.

1.4 Solution

```
echo "S3ViU1RVKGI0czNfNjRfMXNfdGhlX2JhNWk1KQ==" | base64 -d
```

Listing 3: cmd command

2 [Crypto] – Strange sequence of numbers

2.1 Challenge Description

I received a strange document with numbers inside — what could it mean?

2.2 Enumeration & Static Analysis

We are presented with one txt file that we can simply analyze its contents:

```
file strange_sequence_of_numbers.txt
cat strange_sequence_of_numbers.txt
```

Listing 4: initial file checking

The output clearly shows us a list of decimal values:

- **Observation 1:** These values don't exceed 127.

2.3 Exploitation / Solution Path

Since the list of numbers doesn't exceed 127, and the minimum value is 48, we can convert each character from its decimal representation, since all characters are printable.

2.4 Solution

The flag is simply retrieved:

```
flag = "75 117 98 83 84 85 40 97 115 99 49 49 95 99 48 100 51 115 95 97 114 51
    95 97 110 95 49 110 116 101 114 101 115 116 105 110 103 95 119 52 121 95 116
     111 95 103 101 55 95 105 110 116 111 95 99 114 121 112 55 48 103 114 97 112
     104 121 41"

print("".join([chr(int(j)) for j in flag.split(" ")]))
```

Listing 5: exp.py

3 [Crypto] – Unlucky 13

3.1 Challenge Description

13 is an unlucky number. Three layers of encryption, thirteen reasons not to try decrypting this. Something leaked — hopefully it will help you.

3.2 Enumeration & Static Analysis

We are presented with two files, output.txt and encrypt.py. We can analyze their contents:

```
file output.txt
cat output.txt
cat encrypt.py
```

Listing 6: initial file checking

The following observations can be drawn:

- **Observation 1:** From the cat of the file output.txt, we understand that it might be used an algorithm similar to the RSA algorithm, since we are presented with the modulus n, exponent e, and ciphertext c;
- **Observation 2:** From the cat of the encrypt.py we can observe the sequence of encrypting actions that have been taken to encrypt the flag.

3.3 Exploitation / Solution Path

From the encrypt.py, we can observe that the number 13 is used as seed for the pseudo random number generator (prng). In the first layer, the values generated by the prng are xorred with the flag characters; in the second layer, the customized forgotten cipher is applied to the data produced from the first layer. Finally, the value produced from the second layer is converted to a decimal value, and transmitted in RSA fashion, that is $c = m^e mod N$. Since the ciphertext produced is much smaller with respect to the modulus N, we can obtain the value of m by taking the cubic root of c. The forgotten cipher creates a mesh of Substitution boxes driven by the secret key, which is known, and thus fully revertible. Finally, the first layer is also revertible since the seed is known, and we can generate the same sequence of prng sequence.

3.4 Solution

```
import hashlib, math, gmpy2
from decimal import Decimal
from Crypto.Util.number import long_to_bytes,bytes_to_long

```

```
n = 136586330371317880323516184270722474767179545423964086335607738843645545590705114013381313
e = 3
c = 581064029452524128858679080421167948194643057449718995780730203040675435480708074571786581
#FLAG = open("flag.txt", "rb").read().strip()
FLAG=b"0000000000000000"

def cursed_prng(seed, length):
    state = seed
    stream = []
    for _ in range(length):
        state = (state * 1313 + 131313) % (2**32)
        stream.append(state & 0xFF)
    return bytes(stream)

UNLUCKY_NUMBER = 13

layer1 = bytes(a ^ b for a, b in zip(
    FLAG,
    cursed_prng(UNLUCKY_NUMBER, 64)#len(FLAG))
))

def get_flag(layer1):
    res = bytes(a ^ b for a,b in zip(layer1, cursed_prng(UNLUCKY_NUMBER,64)))
    return res

def forgotten_cipher(key, data):
    S = list(range(256))
    j = 0
    for i in range(256):
        j = (j + S[i] + key[i % len(key)]) % 256
        S[i], S[j] = S[j], S[i]
    i = j = 0
    out = []
    for byte in data:
        i = (i + 1) % 256
        j = (j + S[i]) % 256
        S[i], S[j] = S[j], S[i]
        out.append(byte ^ S[(S[i] + S[j]) % 256])
    return bytes(out)

def recover_cipher(key,layer2):
    S = list(range(256))
    j = 0
    for i in range(256):
        j = (j + S[i] + key[i % len(key)]) % 256
        S[i], S[j] = S[j], S[i]
    i = j = 0
    out = []
    #start the recovring
    for byte in layer2:
        i = (i + 1) % 256
        j = (j + S[i]) % 256
        S[i], S[j] = S[j], S[i]
        out.append(byte ^ S[(S[i] + S[j]) % 256])
    return bytes(out)

secret = b"Unlucky" + str(UNLUCKY_NUMBER).encode()
fc_key = hashlib.sha256(secret).digest()[:16]
```

```
64
65 layer2 = forgotten_cipher(fc_key, layer1)
66
67 #m = int.from_bytes(layer2, "big")
68 #c = pow(m, e, n)
69 #m = (c**1/3) % n
70
71 m, exact = gmpy2.iroot(c, 3)
72 if exact:
73     print("Message found:", m)
74 #m = math.cbrt(Decimal(c))
75 #print(m)
76 int_m = int(m)
77 layer1 = recover_cipher(fc_key, long_to_bytes(int_m))
78 FLAG = get_flag(layer1)
79 print(FLAG)
```

Listing 7: exp.py

4 [Crypto] – Nintendo 3DS

4.1 Challenge Description

Hey, I can never remember the name of the algorithm, but it's something very similar to Nintendo 3DS — help me figure out what was written there ^^.

4.2 Enumeration & Static Analysis

As we are presented with a txt file, we can easily analyze its contents:

```
1 file ./3ds_output_2.txt
2 cat ./3ds_output_2.txt
```

Listing 8: initial file checking

The output clearly shows us a list of useful information:

- **Observation 1:** It was used the CBC cipher mode, with a PKCS5 padding scheme;
- **Observation 2:** We are presented with three values (1,2,3), which probably represent the hidden key, encoded in a different manner - string 1 is encoded in base64, characters of value 2 are encoded in decimal form, and string 3 is encoded in hexadecimal;
- **Observation 3:** We are presented with two values for the initialization vectors, ivx and ivm, of which ivx is encoded in hexadecimal and ivm is already in plaintext.

4.3 Exploitation / Solution Path

As the description hints, the cryptographic algorithm involves a triple DES cipher. A 3DES cipher is a 64-bit symmetric block cipher that applies three times the same encryption/decryption algorithm - its key size is 168 bits long (56 x 3), and nowadays its usage is deprecated with respect to the most secure AES algorithm. First we start to decode what is decodable.

```
1 echo "TjFudDNuZG8===" | base64 -d
```

Listing 9: cmd command

```
1 "".join(chr(x) for x in (83, 51, 99, 117, 114, 49, 116, 121))
2 print(bytes.fromhex("4b33792132303236").decode('utf-8'))
3 key = var_1 + var_2 + var_3
```

Listing 10: python commands

The mater key is then obtained from the concatenation of the three values obtained after decoding.

The initialization vector is also decoded, and since it is used the CBC mode, it means that the initialization vector must be at most 64 bits long: this requires us to XOR the two iv strings:

```
ivx = bytes.fromhex("0a001f0273760054")
iv = bytes([a ^ b for a,b in zip(ivx,ivm)])
```

Listing 11: python command

4.4 Solution

```
from Crypto.Cipher import DES3,DES
from Crypto.Util.Padding import pad, unpad
from Crypto.Random import get_random_bytes

var_1 = b"N1nt3ndo"
var_2 = b"S3cur1ty"
#print(bytes.fromhex("4b33792132303236").decode('utf-8'))
var_3 = b"K3y!2026"

key = var_1 + var_2 + var_3

ivx = bytes.fromhex("0a001f0273760054")
ivm  = b"M4r10Br0"
iv = bytes([a ^ b for a,b in zip(ivx,ivm)])
#print(iv) #b'G4m3C4rd'

cipher = bytes.fromhex("072a8e75459a545679f3aa56a9fafb38871022de0c9bd5d"+"7
    ef55e8dad7861662eb0fb630d9cdf9dd8c64a3a8ac28b86a")

cipher_decrypt = DES3.new(key, DES3.MODE_CBC, iv)
decrypted_padded = cipher_decrypt.decrypt(cipher)
# Remove padding
original_data = unpad(decrypted_padded, DES3.block_size, style='pkcs7')

#print(f"Original Message: {original_data}")
print(f"Original Message: {original_data.decode('utf-8')}")
```

Listing 12: exp.py

4.5 Key Takeaways

1. Fun riddle that required us to correlate the data at hand with the encryption algorithm.

5 [Crypto] – Furry Cipher

5.1 Challenge Description

I was scrolling through my email and saw a message from FurryHater_2009) asking to decrypt some strange message, along with two attached files.

5.2 Enumeration & Static Analysis

We are presented with two files - a Furry_Cipher.py, and Weird_Furry_text.txt

```
file ./Furry\Cipher.py
cat ./Furry_Cipher.py
file ./Weird_Furry_text.txt
cat ./Weird_Furry_text.txt
```

```
strings ./Weird_Furry_text.txt
ls -la
```

Listing 13: initial file checking

The output clearly shows us a list of useful information:

- **Observation 1:** The output of cat on Furry_Cipher.py shows us that a personalized algorithm was used for the encryption of the flag string;
- **Observation 2:** The output of cat on Weird_Furry_text.txt shows us that the file is padded with garbage, and there is no point in visually analysing what is there;
- **Observation 3:** Also, the output of strings doesn't help us, as the padding characters are all printable characters.

5.3 Exploitation / Solution Path

The exploitation is divided in two parts. In the first part, the target string must be retrieved from all the garbage that is present in Weird_Furry_text.txt; the second part reverts back the encryption algorithm (and thus recovers the hidden flag).

As we inspect the cipher, we notice that it maps printable characters that fall in the interval [a-zA-Z0-9], to other characters that are in the same interval. Since the characters [()_] remain unchanged, we can add them to the string search [a-zA-Z0-9()_].

```
num = char_map[char]
key_val = key_values[i % 3]
if i % 3 == 0:
    encrypted = (num * 13 + key_val * 7) % 62
elif i % 3 == 1:
    encrypted = (num * 17 + key_val * 3 + 11) % 62
else:
    encrypted = (num * 19 + (key_val ^ 42) + 23) % 62
```

Listing 14: personalized cipher - key selection and addition

For what concerns the encryption algorithm, it can be noticed that the key is a vector of length 3, and its key values fall in the range [0,61] (due to the modulus application). The combination of the key values can be searched with brute force, as it would require 62*62*62 = 186 attempts in total. In order to recover the character mappings, we need to leverage our knowledge about the computation of modular multiplicative inverse. That's possible due to the fact that the greatest common divisor (gcd) between (13 and 62, 17 and 62, and 19 and 62) is equal to 1, which means they are coprime. By computing the extended gcd (egcd) between them, for e.g. between 13 and 62, we obtain:

$$egcd(13, 62) = 1 = a * 62 + b * 13 = 4 * 62 + (-19) * 13$$

we can find a value "b" such that $b*(encrypted - key\_val*7)\%62 = num$. In this case, -19 is the inverse modulo 62 of 13, as -19*13 mod 62 is 1. Since the part related to the key gets cancelled under the hypothesis of correct key values, as a result we will obtain the correct mapping num, which can be correctly inverted.

5.4 Solution

We can search our hidden flag as follows:

```
import re

pattern = re.compile(r"[a-zA-Z0-9()_]")
```

```
res=""
with open("Weird_Furry_text.txt", "r") as file:
    for line in file:
        for match in pattern.finditer(line):
            #print(f"Match found: {match.group()}, Position: {match.span()}")
            res += match.group()
print(res)
```

Listing 15: string recovery

And follows the decryption of the flag:

```
import string

def encrypt_custom(plaintext, key_values):
    alphabet = string.ascii_uppercase + string.ascii_lowercase + string.digits
    char_map = {ch: i for i, ch in enumerate(alphabet)} #A:0
    num_map = {i: ch for i, ch in enumerate(alphabet)}  #0:A
    #print("char_map",char_map)
    #print("num_map",num_map)
    result = []
    for i, char in enumerate(plaintext):
        #index i flows progressively
        #if it is one of them, hust appends 'em
        if char in '()_':
            result.append(char)
        elif char in char_map:
            num = char_map[char]
            key_val = key_values[i % 3]
            if i % 3 == 0:
                encrypted = (num * 13 + key_val * 7) % 62
            elif i % 3 == 1:
                encrypted = (num * 17 + key_val * 3 + 11) % 62
            else:
                encrypted = (num * 19 + (key_val ^ 42) + 23) % 62
            result.append(num_map[encrypted])
        else:
            result.append(char)
    return ''.join(result)

def decrypt(ciphertext,key_values):
    res = []
    index = 0
    alphabet = string.ascii_uppercase + string.ascii_lowercase + string.digits
    char_map = {ch: i for i, ch in enumerate(alphabet)} #A:0
    num_map = {i: ch for i, ch in enumerate(alphabet)}  #0:A
    for i, char in enumerate(ciphertext):
        if char in '()_':
            res.append(char)
        elif char in char_map:
            encrypted = char_map[char]    #index value of the character of the ciphertext wrt to alphabet
            key = key_values[i%3]

            if i%3 == 0:
                num = (encrypted*(-19) - -19*7*key ) % 62
            elif i%3 == 1:
                num = (encrypted*11 - 11*(11 + 3*key) ) % 62
            else:
                num = (encrypted*(-13) - (-13)*(23 + 42^key) ) % 62

            res.append(num_map[num])
        else:
            pass
        index += 1
```

```
    return "".join(res)

if __name__ == "__main__":
    flag = "XiEDJ5(9tV_qY3_v43_t9B3_o9vo_ESM_YR_YA_t_S5t8v_XYL4jt)"
    example_text = "Test123"
    example_key = [1, 2, 3]    #they wrap on 62
    encrypted_example = encrypt_custom(example_text, example_key)
    print(encrypted_example)
    #flag = 'Gfgiyko'


    for i in range(63):
        for j in range(63):
            for k in range(63):
                computed = decrypt(flag,[i,j,k])
                if "KubSTU" in computed:
                    print(decrypt(flag,[i,j,k]))
```

Listing 16: exp.py

6 [Crypto] – Not enough part 1

6.1 Challenge Description

During a system dump, some parameters were damaged: only part of the bits of p were preserved — the last 72 bits were lost. Then, from the recovered parameters, a key was derived by hashing the secret (this also survived — I think it's AES-GCM?)

6.2 Enumeration & Static Analysis

We are presented with one txt file:

```
file output.txt
cat output.txt
```

Listing 17: initial file checking

From the output file, we draw the following conclusions:

- **Observation 1:** target server used the parameters for RSA algorithm (N, e, p_hi);
- **Observation 2:** we are presented with the parameters used for the encryption of the flag through AES-GCM algorithm.

6.3 Exploitation / Solution Path

Only the higher part of the prime is known, but this doesn't stop an attacker from the retrieval of the entire vale of p if it is known at least half sizes of the original p. Since the size of roughly 512 bits, the last 72 bits can be easily retrieved through the Coppersmith attack. The retrieval of the missing part of p is performed in sagemath, while the retrieval of the flag in python.

6.4 Solution

The missing part of p is retrieved by setting up a polynomial ring shifted left by 72, and added the incognito x.

```
N =
    151708532784988710186354895816447243710932251919277531742510058529452761722432439526454252
```

```
p_hi = 261585037973132772577820331336578451283892270218501125702993194784612273676963424289712090

p_known = p_hi << 72
R.<x> = PolynomialRing(Zmod(N))
f = p_known + x
roots = f.small_roots(X=2^72, beta=0.3, epsilon=0.01)
if roots:
    x0 = roots[0]
    p = p_known + Integer(x0)
    print(f"Recovered p: {p}")
    print(f"Factor q: {N // p}")
    print(f"Root value: {x0}")
else:
    print("Could not recover p.")
```

Listing 18: recover.sage

As the prime p is recovered, follows the recovery of all the remaining parameters, and the extraction of the flag:

```
from Crypto.Cipher import AES
from Crypto.Util.number import long_to_bytes,bytes_to_long
import hashlib
import binascii

N = 151708532784988710186354895816447243710932251919277531742510058529452761722432439526454251

e = 65537

p_hi = 261585037973132772577820331336578451283892270218501125702993194784612273676963424289712090

p_lo = 127729287742157157274 7

p = 123530041574450559800504879403701186974161501594698575119589473819974628568320086383998837

q = 122811043250200155554663776035960005780917728411167286397876578361074215172721478413430557

#print(p*q == N)    #true

nonce = bytes.fromhex("8fe6c8d25d0738576b6f6a25")
phi = (p-1)*(q-1)
d = pow(e,-1, phi)
tag = bytes.fromhex("9755d120d8356f29ca31eacff3360ab0")
ciphertext = bytes.fromhex("0094
    dfe5f358aecb96369cf72731d114bf0a0008cbe1d15b98b30f4fd1492e0ee1567a7fd602dc3ff7aa709ea98e7
    ")

kdf = hashlib.sha256(long_to_bytes(d)).digest()[:16]

try:
    cipher = AES.new(kdf, AES.MODE_GCM, nonce=nonce)
    plaintext = cipher.decrypt_and_verify(ciphertext, tag)
    print("Decryption successful:", plaintext.decode('utf-8'))
except ValueError:
    print("Decryption failed: Key is incorrect or message has been corrupted.")
```

Listing 19: exp.py

7 [Crypto] – Not enough part 2

7.1 Challenge Description

During an emergency system recovery, only fragments of two ***-keys were saved. From each prime number, only the high bits survived, and the lower bits were ??????. After recovering the private parameters, a master secret was formed from them, which was then converted via KDF into a key for something.

7.2 Enumeration & Static Analysis

We are presented with one txt file:

```
file output_1.txt
cat output_1.txt
```

Listing 20: initial file checking

From the output file, we draw the following conclusions:

- **Observation 1:** target server used the parameters for the RSA algorithm (N, e, p_hi);
- **Observation 2:** we are presented with the parameters used for the encryption of the flag through AES-GCM algorithm.

7.3 Exploitation / Solution Path

Also this time only the higher part of the prime p is known, so in both cases we are able to retrieve the missing parts. For the first prime p, it can be easily used the previous technique for "Not enough part 1". For the second prime p, it is unknown the length of the missing part. Then, in a for cycle, it can be bruteforced different shifting lengths, until a solution is found.

7.4 Solution

```
N = 
    107693247573615587212525584706568693993099179606983949136202089061406798764011406480235980

p_hi = 
    832313695256042953182732157279176245412116775736305489730893867560396386813292595697421310


for shift_ in range(1,300):
        p_known = p_hi << shift_
        R.<x> = PolynomialRing(Zmod(N))
        f = p_known + x
        roots = f.small_roots(X=2^shift_, beta=0.2, epsilon=0.05)
        if roots:
            x0 = roots[0]
            p = p_known + Integer(x0)
            print(f"Recovered p: {p}")
            print(f"Factor q: {N // p}")
            print(f"Root x0: {x0}")
            break
        else:
            print(f"Could not recover p. for shift: {shift_}")
```

Listing 21: recover.sage

As the prime p is recovered, follows the recovery of all the remaining parameters, and the extraction of the flag:

```python
from Crypto.Cipher import AES
from Crypto.Util.number import long_to_bytes,bytes_to_long
import hashlib
import binascii

N_1 = 829033568076444275112917737613789220294226111883580472215532421220364183868278550491558970

e_1 = 65537
p_1 = 116829345878223159113757772836387227429058414616506292957920099649663493369716988493368630

q_1 = 709610724809317856128092114890620035720076324306526318917435489195825215033099548920069550


phi_1 = (p_1 - 1)*(q_1 - 1)
d_1 = pow(e_1, -1, phi_1)

N_2 = 107693247573615587212525584706568693993099179606983949136202089061406798764011406480235980

e_2 = 65537
p_2 = 100620551621389242148748248478071967952869551264750262994961261962985577950941608549810990

q_2 = 107029076901545110178549371118747759249774780840946402456035107587146459425929485200125030


phi_2 = (p_2 - 1)*(q_2 - 1)
d_2 = pow(e_2, -1, phi_2)

key = long_to_bytes(d_1) + long_to_bytes(d_2)

nonce = bytes.fromhex("492d206feb1c86188589bb46")
tag = bytes.fromhex("02d3cd6b6aaa54d7026029a16fba111b")
ciphertext = bytes.fromhex("4bbbb06c1b54b2bca2eab8f6cd2e7bb90f269bf7f8a7661ef84f78a554bbc6dca350f70e2f0df47add8bca0a13")

kdf = hashlib.sha256(key).digest()[:16]

try:
    cipher = AES.new(kdf, AES.MODE_GCM, nonce=nonce)
    plaintext = cipher.decrypt_and_verify(ciphertext, tag)
    print("Decryption successful:", plaintext.decode('utf-8'))
except ValueError:
    print("Decryption failed: Key is incorrect or message has been corrupted.")
```

Listing 22: exp.py